

Отчет по лабораторной работе №8

Дисциплина: Архитектура ЭВМ

Тема: Программирование цикла. Обработка аргументов командной строки.

Студент: Али Мухтар Хадир

Группа: НПИ-01

1. Цель работы

Приобретение навыков написания программ с использованием циклов и обработкой аргументов командной строки. Изучение организации стека в процессоре.

2. Выполнение работы

2.1. Реализация циклов и использование стека

Я создал файл `lab8-1.asm` и реализовал программу, которая выводит числа от N-1 до 0. Чтобы цикл `loop` работал корректно при изменении регистра `ecx` внутри него, я использовал команды `push` и `pop` для сохранения счетчика в стеке.

Листинг 8.1 (lab8-1.asm):

```
%include 'in_out.asm'
```

```
SECTION .data
```

```
    msg1 db 'Введите N: ',0h
```

```
SECTION .bss
```

```
    N: resb 10
```

```
SECTION .text
```

```
    global _start
```

```
_start:
```

```
mov eax,msg1
```

```
call sprint
```

```
mov ecx, N
```

```
mov edx, 10
```

```
call sread
```

```
mov eax,N
```

```
call atoi
```

```
mov [N],eax
```

```
mov ecx,[N]
```

```
label:
```

```
push ecx ; Сохранение счетчика в стек
```

```
sub ecx, 1
```

```
mov [N], ecx
```

```
mov eax, [N]
```

```
call iprintLF
```

```
pop ecx ; Восстановление счетчика из стека
```

```
loop label
```

```
call quit
```

Команда: `nano lab8-1.asm`

Команда: `i686-linux-gnu-ld -o lab8-1 lab8-1.o qemu-i386`

Команда: `./lab8-1`

Ввод числа: `Enter N: 5`

Вывод программы:

`4`

`3`

2
1
0

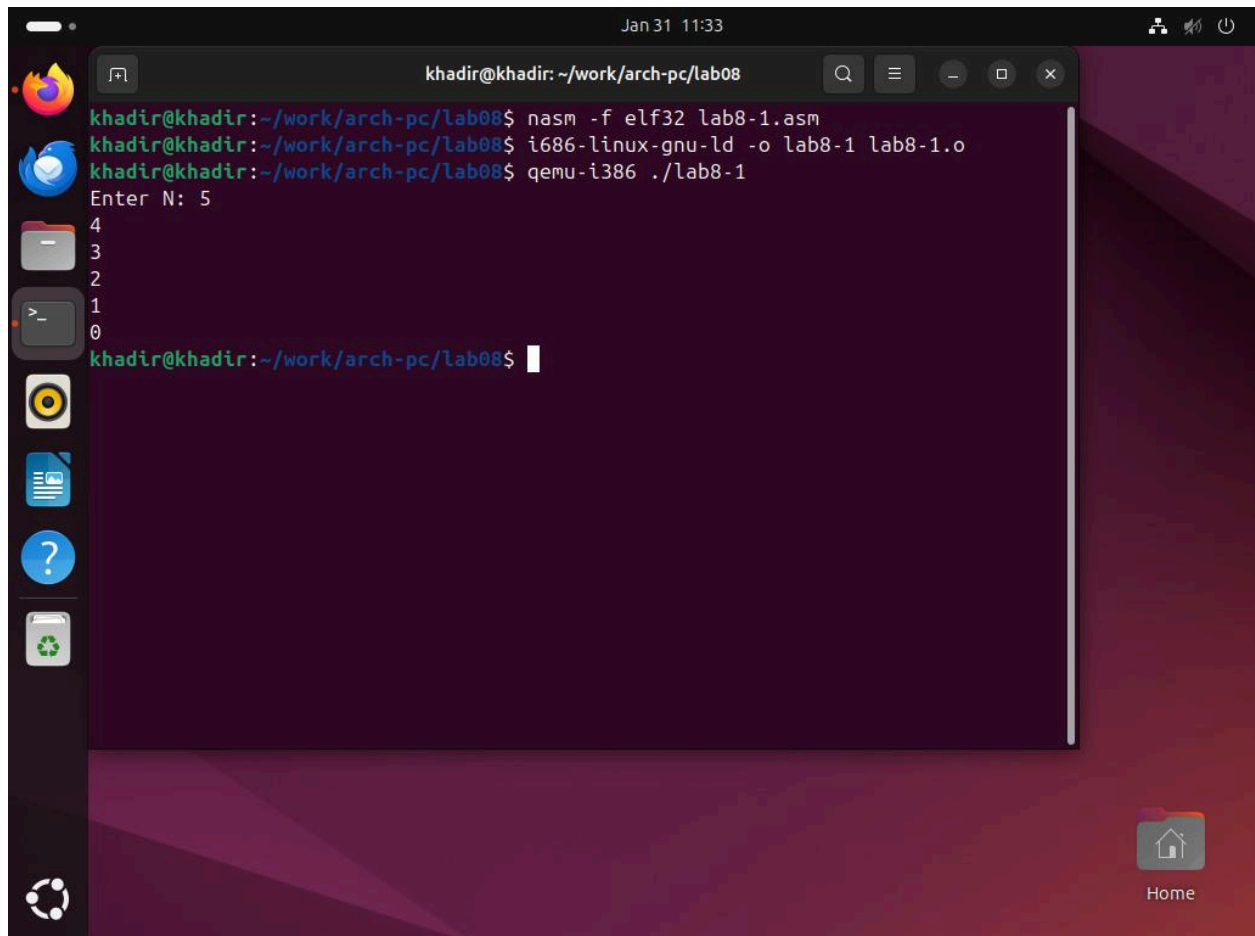


Рис 8.1. Выполнение программы `lab8-1.asm`, демонстрирующее правильное использование стека для сохранения счетчика цикла.

2.2. Обработка аргументов командной строки

Я создал программу `lab8-2.asm`, которая извлекает аргументы из стека и выводит их на экран.

Команды: `nasm -f elf32 lab8-2.asm`

Команды: `i686-linux-gnu-ld -o lab8-2 lab8-2.o`

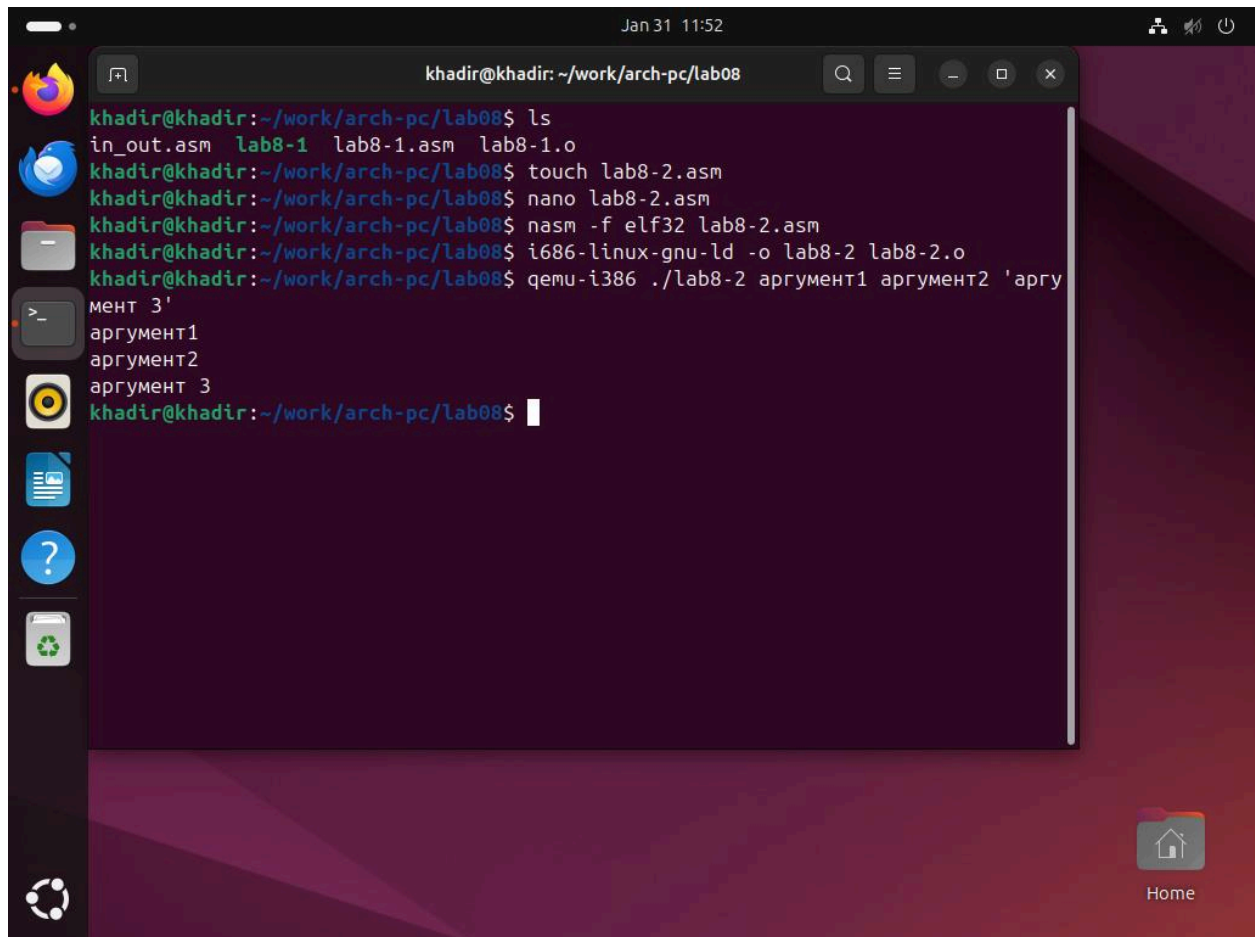
Команды: `qemu-i386 ./lab8-2 аргумент1 аргумент2 'аргумент 3'`

Ожидаемый результат:

аргумент1

аргумент2

аргумент 3



The screenshot shows a terminal window titled 'khadir@khadir: ~/work/arch-pc/lab08' with a search bar and window controls. The terminal output is as follows:

```
khadir@khadir:~/work/arch-pc/lab08$ ls
in_out.asm  lab8-1  lab8-1.asm  lab8-1.o
khadir@khadir:~/work/arch-pc/lab08$ touch lab8-2.asm
khadir@khadir:~/work/arch-pc/lab08$ nano lab8-2.asm
khadir@khadir:~/work/arch-pc/lab08$ nasm -f elf32 lab8-2.asm
khadir@khadir:~/work/arch-pc/lab08$ i686-linux-gnu-ld -o lab8-2 lab8-2.o
khadir@khadir:~/work/arch-pc/lab08$ qemu-i386 ./lab8-2 аргумент1 аргумент2 'аргумент 3'
аргумент1
аргумент2
аргумент 3
khadir@khadir:~/work/arch-pc/lab08$
```

The desktop background is a dark purple/red gradient. On the left is a vertical dock with icons for Firefox, Telegram, a file manager, a terminal, a media player, a help icon, a trash can, and a refresh icon. On the bottom right is a 'Home' button with a house icon.

Рис. 8.2. Обработка и вывод аргументов «аргумент1», «аргумент2» и «аргумент 3».

2.3. Вычисление произведения аргументов

Я изменил программу из Листинга 8.3 для вычисления произведения чисел, переданных в качестве аргументов.

Листинг 8.2 (lab8-3.asm):

```
%include 'in_out.asm'
```

```
SECTION .data
```

```
msg db "Результат произведения: ",0
```

```
SECTION .text
```

```
global _start
```

```
_start:
```

```
pop ecx
```

```
pop edx
```

```
sub ecx, 1
```

```
mov esi, 1
```

```
next:
```

```
cmp ecx, 0
```

```
jz _end
```

```
pop eax
```

```
call atoi
```

```
imul esi, eax
```

```
loop next
```

```
_end:
```

```
mov eax, msg
```

```
call sprint
```

```
mov eax, esi
```

```
call iprintLF
```

call quit

Команды: `nasm -f elf32 lab8-3.asm`

Команды: `i686-linux-gnu-ld -o lab8-3 lab8-3.o`

Команды: `qemu-i386 ./lab8-3 2 3 4`

Ожидаемый результат: Результат: 24

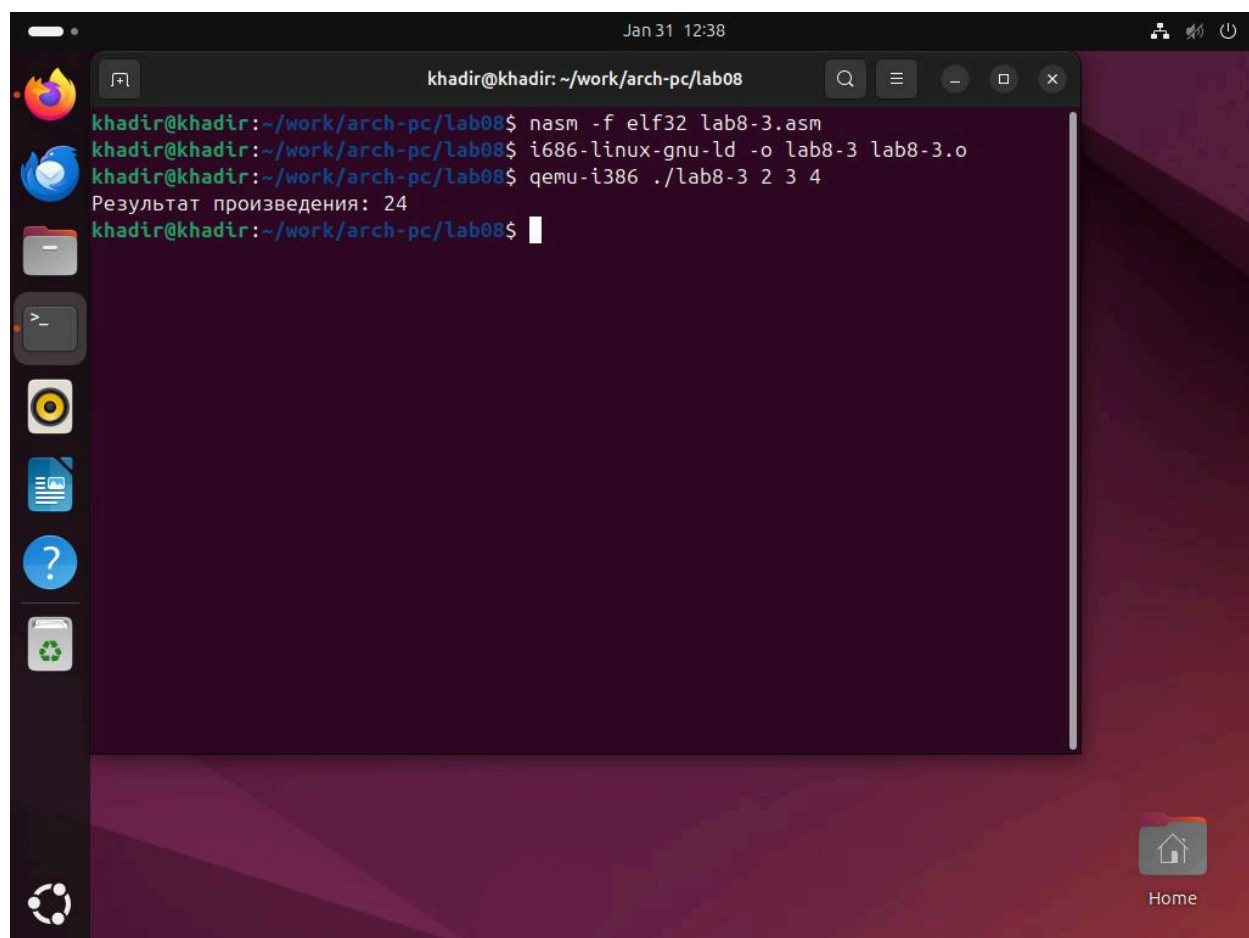


Рис. 8.3. Вычисление произведения аргументов 2, 3 и 4.

3. Самостоятельная работа (Вариант 12)

Я написал программу для вычисления суммы значений функции $f(x)=15x-9$ для набора x , переданных как аргументы.

Листинг 8.3 (lab8-4.asm):

```
%include 'in_out.asm'
```

```
SECTION .data
```

```
msg_f db "Функция:  $f(x) = 15x - 9$ ",0
```

```
msg_r db "Результат: ",0
```

```
SECTION .text
```

```
global _start
```

```
_start:
```

```
pop ecx
```

```
pop edx
```

```
sub ecx, 1
```

```
mov esi, 0
```

```
next:
```

```
cmp ecx, 0
```

```
jz _end
```

```
pop eax
```

```
call atoi
```

```
mov ebx, 15
```

```
imul eax, ebx
```

```
sub eax, 9
```

```
add esi, eax
```

```
loop next
```

```
_end:
```

```
mov eax, msg_f
```

```
call sprintLF
```

```
mov eax, msg_r
```

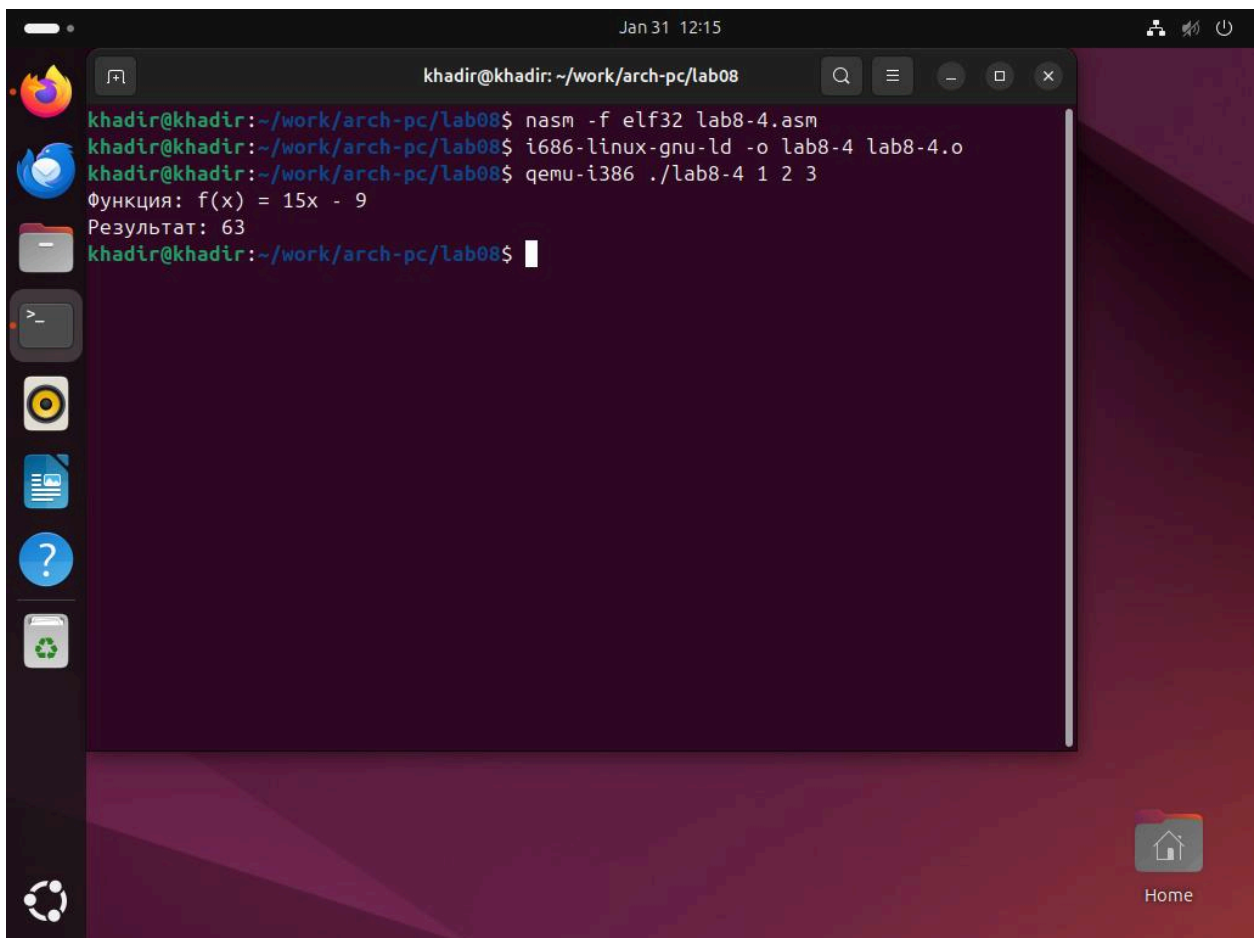
```
call sprint
```

```
mov eax, esi
```

```
call iprintLF
```

```
call quit
```

- Команды: `nasm -f elf32 lab8-4.asm`
- Команды: `i686-linux-gnu-ld -o lab8-4 lab8-4.o`
- Команды: `qemu-i386 ./lab8-4 1 2 3`
- Ожидаемый результат: Результат: 63



The screenshot shows a terminal window titled "khadir@khadir: ~/work/arch-pc/lab08" with a search bar and window controls. The terminal output is as follows:

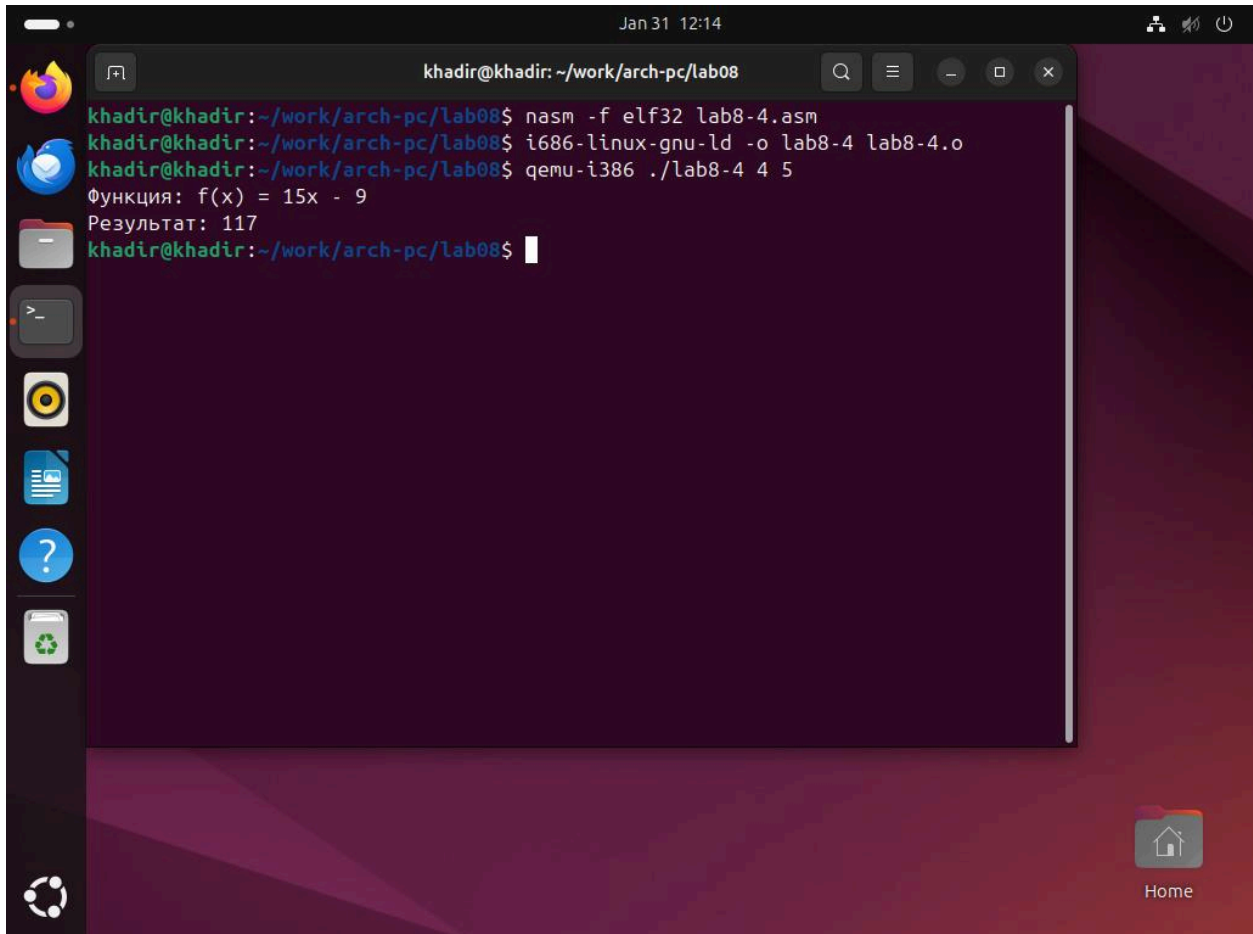
```
khadir@khadir:~/work/arch-pc/lab08$ nasm -f elf32 lab8-4.asm
khadir@khadir:~/work/arch-pc/lab08$ i686-linux-gnu-ld -o lab8-4 lab8-4.o
khadir@khadir:~/work/arch-pc/lab08$ qemu-i386 ./lab8-4 1 2 3
Функция: f(x) = 15x - 9
Результат: 63
khadir@khadir:~/work/arch-pc/lab08$
```

The desktop background is a dark purple/red gradient. On the left is a vertical dock with icons for Firefox, Telegram, a file manager, a terminal, a media player, a document, a help icon, and a trash can. At the bottom right is a "Home" button with a house icon.

Рис. 8.4. Вычисление суммы значений функции $15x-9$ для аргументов 1, 2 и 3.

После: `qemu-i386 ./lab8-4 4 5`

Ожидаемый результат: **Результат: 117**



```
khadir@khadir: ~/work/arch-pc/lab08
khadir@khadir:~/work/arch-pc/lab08$ nasm -f elf32 lab8-4.asm
khadir@khadir:~/work/arch-pc/lab08$ i686-linux-gnu-ld -o lab8-4 lab8-4.o
khadir@khadir:~/work/arch-pc/lab08$ qemu-i386 ./lab8-4 4 5
Функция: f(x) = 15x - 9
Результат: 117
khadir@khadir:~/work/arch-pc/lab08$
```

Рис. 8.5. Вычисление суммы значений функции $15x-9$ для аргументов 4 и 5.

4. Контрольные вопросы

1. Опишите работу команды `loop`. Команда `loop` использует регистр `ecx` как счетчик. Она уменьшает `ecx` на 1. Если `ecx` не равен 0, программа возвращается к метке. Если `ecx` равен 0, цикл завершается.

2. Как организовать цикл с помощью команд условных переходов? Цикл можно сделать без `loop`. Нужно использовать команду уменьшения `dec ecx`, затем сравнение `cmp ecx, 0` и команду перехода `jne label` (прыжок, если не ноль).

3. Дайте определение понятия «стек». Стек — это специальная область памяти для временного хранения данных. Он работает по принципу LIFO (Last In — First Out): "последним пришел — первым ушел".

4. Как осуществляется порядок выборки данных из стека? Данные извлекаются в обратном порядке. Элемент, который вы положили в стек последним (командой `push`), будет первым, который вы заберете (командой `pop`).

Выводы

В ходе работы я изучил механизмы работы циклов в NASM и использование стека для временного хранения данных. Также я научился обрабатывать аргументы командной строки, что позволяет создавать более гибкие программы.